CS338 - Introduction to the Theory of Computation

Spring 2026

Lecture #0

➢ Introduction

Prof Pietro S. Oliveto
Department of Computer Science and Engineering
Southern University of Science and Technology
(SUSTech)

olivetop@sustech.edu.cn
https://faculty.sustech.edu.cn/olivetop

Office: Room 113

➢ About Me

- Previously **Professor of Computer Science, Chair in Algorithms**, University of Sheffield, UK
- Moved to **SUSTech** in April 2023 (Room 113)
- Taught **Algorithms and Data Structures** in the UK for a few years
- Introduced **Theory of Computation** at SUSTech two years ago
- Expertise in **time complexity of randomised search heuristics and bio-inspired computation** (evolutionary algorithms, genetic algorithms, artificial immune systems etc.)
- I am building a **Theory of AI lab** at SUSTech (Room 348B)

Webpages:

- https://faculty.sustech.edu.cn/olivetop
- https://peteroliveto.github.io

➢ What this is about

- **Theoretical foundations** of computer science
- Computability theory **(Part I)**
  - **Understanding** "what is computable"?
  - **Computational models of increasing complexity**
  - What problems **can** be solved in principle and **which** cannot be solved independent of the amount of available resources
- Computational complexity **(Part II)**
  - What is computable **efficiently?**
  - What problems **can** be solved with reasonable time and space and which **cannot**
  - Sorting is an **easy** problem, while scheduling seems much **harder**
  - *What makes problems computationally harder than others?*
- Some of the most important (open) problems in computer science

➢ At the end of this course you'll be able to...

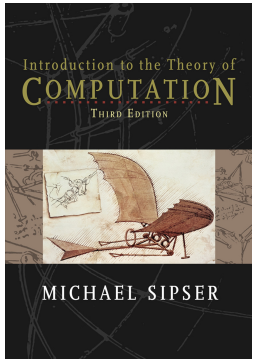
- Analyse rigorously whether a computational problem is **solvable** or not by different computational models
- Describe the **classes of problems** that different models of computation can solve
- Provide examples of problems that **no** reasonable computational model can solve
- Show whether a computational problem can be solved **efficiently**
- Show when a problem is **at least as hard** as another
- Prove to which **computational complexity class** a problem belongs to (i.e., how hard the problem is).

Why care about Theory of Computation?

- Living in the **Age of Algorithms**
- Programming → Computer **Science**
- "*There is nothing so practical as a good theory.*"
- Can an algorithm even **exist** that can solve your problem?
- Can any algorithm solve your problem within **reasonable time and space** (i.e., memory)?
- Appreciate when to **relax your goals** e.g., *approximation algorithms or fixed parameter tractable algorithms*, or consider alternative types of computation eg., *randomized algorithms.*
- **Motivation** for more advanced courses CS208 (Algorithms Design and Analysis), and CSE5003/CS5018 (Advanced Algorithms); cryptography requires computational problems that are hard.
- **Employability** – Tencent, Huawei, Alibaba etc. require eg., NP-hardness.
- Things that every computer scientist is expected to know

Textbook

- **Michael Sipser (2013) *Introduction to the Theory of Computation,* third edition (Cengage Learning)**
  - Available in library
  - Used for the same course at MIT
  - We will build upon those slides



Teaching

- **Lectures**
  - **Tuesday 16:20-18:10**, Lecture Hall 1, Room **403**
  - mixture of slides, notes, discussions, etc.
  - Blackboard: https://bb.sustech.edu.cn (check regularly)
- **Tutorial / Lab sessions**
  - **Tuesday 19:00-20:50**, Lecture Hall 1, Room **403**
  - **Marked exercises**: practicing the theory learned at lecture
  - After submission, the best provided solutions will be presented
  - **Ask questions**, get **feedback**, get help for current exercise sheet

Assessment

- Attendance: **10%** of module mark
- Marked Lab exercise sheets : **40%** of module mark
  - Test your understanding of the lecture contents
  - Help you **prepare for final exam**
  - Find out in labs whether your solution was correct.
  - Exercise deadline day will be on each sheet; Submit by **Tuesday at 4pm**
  - Solutions will be released after sheets were discussed in labs.
- Final exam: **50%** of module mark
  - Written test: 2 hours
- We'll provide a revision class and a mock exam.

Grading Policy

- Strict late assignment policy (no late submissions allowed)
- Write your assignment on your own
- ZERO tolerance on plagiarism (Software will be used to detect plagiarism and cases reported to the university)

Tentative Schedule

No	Topic
1	DFA & Regular Languages
2	NFA
3	Regular Expressions
4	Non Regular Languages
5	PDA & Context Free Languages
6	Non Context Free Languages
7	Turing Machines
8	Non Deterministic TM

No	Topic
9	Decidable & Recognisable Languages
10	Undecidable Languages
11	The Reducibility Method
12	P Complexity Class
13	NP Complexity Class
14	NP Completeness
15	Cook-Levin Theorem
16	Revision Class

Expectations on you

- In class:
  - Attend lectures and participate (ask questions, take notes)
  - Attend labs and participate (ask questions, you can also present solutions)
- Outside class:
  - **READ THE BOOK**
  - Recap slides/notes after each lecture
  - Do all exercise sheets
  - Use the discussion board on Blackboard to ask questions
- Office hour (Office 113 - just drop in):
  - **Friday 14:00-16:00**
  - or by appointment: olivetop@sustech.edu.cn